Conserver les preuves, borner le contexte du pipeline LANCE

Diagnostic actuel et protocole d'évaluation

Projet LANCE

Dernière révision : 1er octobre 2026 à 10:23
Europe/Brussels (UTC+02:00)

Exploration rouverte ; architecture proposée

Résumé. La gestion du contexte concerne la conservation des observations autant que la fenêtre du modèle. La revue des six phases identifie des coupes avant archivage, des historiques sans budget d'entrée, des projections parfois sans récupération et des transmissions qui retirent certaines alternatives. Cinq tests diagnostiques synthétiques caractérisent des pertes d'information et des limites de reprise ; leur effet sur les scores réels reste inconnu. L'architecture proposée sépare une observation archivée de ses vues temporaires, rend les omissions explicites et borne les lectures ciblées. Le protocole compare d'abord une sélection déterministe renforcée à des représentations augmentées par le modèle, à capacités de lecture identiques. Aucune amélioration de rappel, coût ou réussite réseau n'est établie. L'utilité du mémo et de la rédaction doit être évaluée séparément des constats candidats du scanner.

Table des matières

1 Objectif, périmètre et méthode **3**

2 Capture et historique : deux limites différentes **4**

3 Projections et reprises : conserver moins, retrouver quoi ? **5**

4 Transmission, attribution et isolation **6**

5 Résultats locaux disponibles et ce qu'ils ne prouvent pas **7**

6 Architecture proposée : une preuve, plusieurs vues **8**

7 Protocole proposé : isoler les causes du changement **9**

8 Décisions à prendre et limites de lecture **10**

9 Conclusion et prochaines étapes **10**

Références **11**

1 Objectif, périmètre et méthode

La difficulté ne se réduit pas à la fenêtre du modèle. Le code présente des pertes possibles *avant* archivage, des conversations outils sans budget d'entrée, des projections sans accès de récupération et des transmissions entre phases qui retirent certaines alternatives. Un plafond de sortie plus élevé ne répare pas ces pertes.

La priorité proposée est de préserver la capture native, rendre les omissions observables et comparer des représentations récupérables. Cette priorité est une conclusion de conception issue du code ; **aucun gain de rappel, coût ou réussite réseau n'est démontré**. Les cinq probes locales caractérisent les transformations, sans appeler de modèle ni de cible.

1.1 Repères de lecture

Un grand modèle de langage (LLM) reçoit une requête contenant des consignes et un contexte : messages, observations et résultats d'outils. Sa fenêtre limite la quantité d'entrée et de sortie disponible, mesurée en tokens (unités de texte propres au modèle), distincts des caractères et des octets.

La capture native est la sortie de l'outil avant réduction ; le journal conserve les résultats archivés ; une projection est la vue réduite envoyée au modèle. Une référence de journal identifie une observation conservée, sans garantir que son contenu prouve une faille. Les *probes* sont des tests diagnostiques sur des entrées synthétiques. Un run est une exécution de l'audit.

Les profils `compact` et `full` sont des configurations qui changent représentation, outils et budgets. D1 désigne l'automatisation à règles sans LLM ; A1 l'agent avec LLM. Leur comparaison actuelle impose `full`. Les codes C1–C8 repèrent les constats, P1–P5 les tests synthétiques du tableau 1, et R0–R3 les représentations proposées. Les priorités P0–P2 de la section 6 sont un classement de travaux, indépendant de ces tests.

1.2 Méthode et base inspectée

La révision source étudiée est `45c8dab892ecb4e5f513813387090e0cded1b85d`. Le checkout contenait déjà une réorganisation documentaire ; le SHA ne représente donc pas seul tous les documents lus. La revue combine lecture du code, contrats actuels, probes synthétiques et sources primaires ciblées. Les corrections proposées ne sont pas implémentées.

L'ancien `plan.md` de ce dossier reste archivé. Il concernait surtout la rédaction globale du rapport ; ses estimations et anciens numéros de phase ne décrivent plus l'architecture actuelle. Les notes détaillées sont conservées dans `analysis.md`. Dans les références de code ci-dessous, `analysis/`, `verification/`, `intrusion/` et `report/` désignent les sous-dossiers de `src/agent/phases/`.

1.3 Acquis à préserver

Frontière	Comportement vérifié dans le code
Conversations	Chaque invocation de la boucle outils démarre un historique indépendant ; les requêtes successives de cette boucle reprennent cet historique. Il n'existe pas une conversation unique transmise des phases 1 à 6.
Analyse et preuve	Phase 3 par équipement ; phase 4 par hypothèse et équipement. La preuve de phase 3 reste un contexte, pas un verdict de vérification.
Traçabilité	Journal avec références, arguments, phase, agent et origine ; les livrables sont liés explicitement au dossier du run.
Rapport	Phase 6 par fiches, cache et reprises bornées ; assemblage et synthèse factuelle déterministes. Les commentaires ne sont pas des preuves validées.

Preuve / limite. `src/agent/provider.py:415` ; `analysis/run.py:1092--1128` ; `verification/run.py:137--173` ; `src/agent/core/executor.py:49--100` ; `report/sections.py:97--220`.

2 Capture et historique : deux limites différentes

La figure 1 localise deux endroits où une information peut disparaître. Une réduction avant archivage et une vue abrégée d'une source conservée n'offrent pas la même possibilité de réparation. Ce schéma décrit des transformations présentes ; il ne signifie pas que chaque outil emprunte tous ces chemins.



Figure 1 – État courant : le chemin YAML peut archiver un résultat déjà filtré ou tronqué. Les projections ultérieures peuvent encore réduire ce résultat. Les références existantes ne recréent pas un brut jamais conservé.

2.1 C1 — Capture native perdue avant journalisation

Priorité P0. Le chargeur YAML filtre les lignes puis applique `stdout[:max_output]` avant de retourner son résultat. Le wrapper d'exécution archive ce retour. Le journal peut donc conserver intégralement le *résultat transformé*, sans conserver le stdout natif. Les plafonds déclarés comprennent 4 000 caractères pour HTTP GET et nmap, 2 000 pour SSH exec.

Le suffixe `[truncated]` indique une coupure, mais ne rend pas son suffixe récupérable. Ce constat porte sur le chemin de chargement inspecté ; il ne doit pas être étendu à tout le catalogue d'outils sans vérifier leurs gestionnaires. L'effet sur le rappel réel reste une hypothèse. Une expérience de compression exige une capture de référence complète, ou doit déclarer qu'elle est indisponible.

Preuve / limite. `src/agent/tools/tool_loader.py:228--243` ; `src/agent/core/executor.py:128--132`.

2.2 C2 — Historique croissant sans admission par taille

P0 pour la mesure, P1 pour la correction. La boucle générique ajoute réponses et résultats à `messages`, puis renvoie la liste entière au fournisseur. La limite de 2 000 caractères concerne l'événement d'interface, pas le contenu transmis. `read_deliverable` renvoie également un fichier entier. Les profils fixent surtout tours et sortie : reconnaissance jusqu'à 50 tours, intrusion full jusqu'à 80.

Aucune admission par budget d'entrée, réserve de contexte ou lecture paginée n'a été trouvée dans cette boucle. Le routage automatique compact/full dépend du nombre de paramètres déclaré ; il ne mesure pas la fenêtre effectivement disponible. Cela établit un risque de requêtes volumineuses, *pas* la cause des timeouts observés ailleurs. Une fin `length` indique une troncature de sortie ; elle ne démontre pas une saturation de l'entrée.

Preuve / limite. `src/agent/provider.py:647,986,1181--1182` ; `src/agent/tools/deliverable.py:139--140` ; `src/agent/execution_profiles.py:100--141,281--298`.

3 Projections et reprises : conserver moins, retrouver quoi ?

3.1 C3 — Compact peut retirer une preuve sans recours

Priorité P1. La projection de phase 3 borne chaque résultat à environ 700 caractères en tête et queue, et le cumul des entrées à 5 000 caractères. Elle suit l'ordre d'insertion ; des services entiers peuvent disparaître. Le compteur `omitted_entries` couvre les entrées supprimées, pas chaque résultat raccourci ni le volume retiré à l'intérieur d'une entrée.

Le plafond ne couvre pas les findings déterministes et les voisins ajoutés à côté. Il ne constitue donc pas un budget global du prompt. Surtout, la branche locale compacte mentionne l'artefact de scan mais reçoit `tools=[]`. Cette référence est utilisable par un lecteur humain ; le modèle ne peut pas la consulter pendant cet appel.

Le même écart existe dans la récupération par blocs : les scans sérialisés sont coupés à 12 000 caractères, avec renvoi vers l'artefact complet, alors que seule la sauvegarde est exposée. Une coupe de chaîne peut aussi interrompre la représentation JSON. L'agent doit finaliser avec ce qu'il voit, même si la référence indique qu'une information supplémentaire existe.

Preuve / limite. `analysis/compact.py:15--69` ; `analysis/run.py:442--444,481--484,1142--1184`. Les probes P1 et P2, section 5, caractérisent ces transformations sans mesurer un score de détection.

3.2 C4 — Récupération bornée en appels, pas en difficulté

Priorité P1. La reprise de phase 3 garde les 16 dernières observations et les 1 200 premiers caractères de chacune. Les enregistrements conservés contiennent l'outil, les arguments et le résultat ; ils ne portent pas la référence du journal ni un indicateur de cette coupe. La représentation finale est limitée à 6 000 caractères. Son classement utilise les ports ou services trouvés dans le nom de l'outil et son résultat, sans exploiter les arguments pour ce score.

Cette politique peut retirer une observation initiale, la fin décisive d'un résultat ou un élément négatif. L'arrêt au premier élément qui ne rentre pas peut également empêcher l'ajout d'éléments suivants plus courts. Ce sont des comportements déterministes dépendant de l'ordre, dont l'impact sur une hypothèse reste à établir.

Le découpage vise deux services par bloc, avec quatre blocs maximum. Les services supplémentaires sont fusionnés dans le dernier bloc. Pour un équipement synthétique de 20 services, la distribution est `[2,2,2,14]`. Le nombre d'appels est borné, mais le dernier sous-problème peut rester volumineux. Les preuves non attribuables à un service sont conservées dans plusieurs blocs, ce qui peut augmenter leur poids.

Preuve / limite. `analysis/block_recovery.py:104--151,195--262`. Les probes P3 et P4 reproduisent ces propriétés.

3.3 La frontière de validation reste nécessaire

Le pipeline actuel ne promeut pas arbitrairement une analyse tronquée. Il distingue `finish_reason=length` de l'absence de livrable et assemble les blocs seulement après validation. Les reprises utilisent les preuves existantes sans relancer de scan. Cette protection doit rester en place lors d'une nouvelle politique de contexte.

La proposition consiste à comparer une meilleure sélection déterministe, un découpage par poids des preuves et des lectures ciblées. Elle ne présuppose pas qu'un résumé LLM serait meilleur. Les tâches multi-services ou multi-équipements demandent en outre une évaluation propre : trop découper peut empêcher une corrélation utile.

Preuve / limite. `analysis/run.py:1312--1332` ; `analysis/block_recovery.py:91--101`. Un succès de validation structurelle ne certifie pas que la sélection a préservé tout le sens pertinent.

4 Transmission, attribution et isolation

4.1 C5 — Alternatives retirées entre phases

P1. Phase 4 reçoit les 500 premiers caractères de `evidence`, sans marqueur de coupe. Les champs structurés et le plan peuvent compenser certaines pertes ; leur présence interdit de conclure que toute attribution disparaît. Phase 5 garde une meilleure entrée par IP, 200 ou 400 caractères de preuve et les 30 premiers identifiants récupérés. Les omissions correspondantes ne sont pas comptées, tandis que les listes de cibles et chaînes n'ont pas de plafond global explicite.

Sur le chemin **full**, le prompt décrit cette source comme complète, demande une lecture unique du contexte et interdit le gros livrable précédent ; le guidage injecté autorise pourtant les lectures des phases 3/4. Le compact local remplace guidage et template : cette contradiction ne lui est pas automatiquement applicable.

Preuve / limite. `verification/run.py:169--172` ; `intrusion/run.py:644--718,814--828` ; `src/agent/prompts/intrusion.txt:25--41` ; `src/agent/core/runner.py:317,327,383`.

4.2 C6 — Le mémo compact local n'ajoute pas de findings

P0 pour l'interprétation. Dans `_uses_compact_local_moe()`, le scanner écrit le JSON canonique, puis le modèle rédige un mémo Markdown. Les chemins inspectés d'agrégation, vérification et rapport ne consomment pas ce mémo pour ajouter des findings. Le score de détection de cette branche ne mesure donc pas un apport de nouvelles hypothèses issues du mémo.

Cette limite **ne caractérise pas directement D1/A1**, dont le contraste actuel exige full. Elle concerne un autre mode d'exécution. Un mémo jugé inutilisable peut néanmoins faire échouer l'étape d'analyse, alors que le résultat scanner existe. L'appel local ne demande pas les métadonnées de complétion ; son contrôle de texte reste heuristique.

Preuve / limite. `analysis/run.py:1162--1216` ; `analysis/compact.py:72--180` ; `src/agent/core/memo.py:5--26`. Séparer validité canonique, complétude agent et qualité rédactionnelle.

4.3 C7 — État de contexte encore global

P1 avant concurrence. Graphe, topologie et mode découverte restent dans des variables de module ; le voisinage de phase 3 y est lu. La politique CVE et les filtres de skills sont également globaux. L'isolation des livrables ne garantit donc pas celle de toutes les dépendances de deux Pipeline dans un même processus. L'existence d'un incident réel n'est pas établie : il faut vérifier les modes déployés et tester un entrelacement A/B.

Preuve / limite. `src/agent/tools/graph_tools.py:31--54` ; `src/agent/tools/skill_tools.py:31--37,63--75` ; `analysis/run.py:1113--1115`.

4.4 C8 — Rapport : bonne frontière factuelle, prose à évaluer

P2. Les fiches conservent des extraits de 1 600 caractères avec référence et empreinte. Une fiche supérieure à 24 000 octets perd son commentaire, pas ses faits. Le filtre de contradiction ne vise que quelques expressions anglaises et reçoit un contexte vide : il peut manquer une contradiction ou rejeter une expression pourtant étayée. Aucune fréquence n'est mesurée. Les commentaires sont déjà marqués non validés ; leur utilité doit être évaluée séparément du score d'audit.

Preuve / limite. `report/sections.py:132--148` ; `report/run.py:95--96,136--138` ; `report/validation.py:4--20`.

5 Résultats locaux disponibles et ce qu'ils ne prouvent pas

5.1 Cinq probes synthétiques reproductibles

Les probes utilisent seulement la bibliothèque standard, une extraction AST de la fonction de projection et les modules purs de récupération et de mémo. Elles n'importent pas Pipeline et ne déclenchent ni fournisseur ni scanner. Les assertions décrivent la version étudiée ; elles ne sont pas les critères d'un futur correctif.

ID	Entrée synthétique	Observation reproduite
P1	Un marqueur entre deux segments de 2 000 caractères.	Marqueur central absent après projection ; `omitted_entries=0`. Le compteur n'annonce pas cette perte interne.
P2	20 services, un résultat de 1 000 caractères chacun.	6 services conservés ; 14 entrées omises. Le cas établit une dépendance au budget et à l'ordre.
P3	17 observations, chacune avec marqueur après 1 300 caractères.	16 observations conservées ; tous les marqueurs tardifs absents ; 1 200 caractères par résultat, sans référence de journal dans cette copie.
P4	Un équipement décrivant 20 services.	Quatre blocs de tailles 2, 2, 2 et 14 : la borne d'appels ne borne pas la taille de chaque bloc.
P5	Deux phrases interrompues, en français et en anglais.	Ni « nécessite une » ni « caused by an » ne déclenche l'heuristique de mémo inutilisable.

Table 1 – Mesures de caractérisation logicielle sur des entrées construites. Aucun taux de faux négatifs ou de qualité LLM n'en découle.

Reproduction depuis la racine du dépôt :

```
python3.12-B
benchmarks/experiments/research-review/probe-context.py
```

Les deux lignes ci-dessus forment une seule commande. Le script et ses entrées sont conservés ; il affiche la réussite des cinq probes.

5.2 Validation logicielle consolidée

Le journal `research/2026-09-27-agent-vs-automation/validation.md` consigne **381 tests ciblés réussis en 8,35 secondes**, après préparation d'un environnement Python 3.12.2 et pytest 9.1.1 temporaire. La sélection couvre aussi l'évaluation, le chargeur d'outils, les fiches, les blocs et les statuts. Les fournisseurs et outils sont simulés ; la suite complète n'a pas été exécutée. Le détail des dépendances et tentatives initiales reste dans ce journal, sans les confondre avec des régressions.

5.3 Artefacts historiques exclus des performances

Les exemples `2026-09-11_185118`, `2026-09-12_172518` et `2026-09-12_203157`, sous `output/agent/`, portent d'anciens modes de rapport. Leurs métadonnées ne documentent pas ensemble révision, modèle, fournisseur et résultats globaux. Des notes répétitives peuvent provenir de fixtures ou probes.

Le fichier de contexte de 981 930 octets du premier exemple ne prouve pas que ce volume a été envoyé au modèle : la métadonnée d'entrée indique 15 319 octets. Ces dossiers ne permettent ni un taux actuel d'échec ni l'attribution d'un timeout au contexte. Le récit documentaire d'une reprise de neuf sections le 16 septembre reste un résultat rapporté ; ses preuves primaires n'ont pas été inspectées ici.

6 Architecture proposée : une preuve, plusieurs vues

La figure 2 distingue l'observation durable et sa présentation temporaire. Une observation archivée n'est pas automatiquement une preuve validée. La projection demeure révocable : elle peut être reconstruite ou complétée sans refaire une action réseau. Le modèle n'obtient pas pour autant un accès illimité ; les lectures et l'entrée restent bornées.

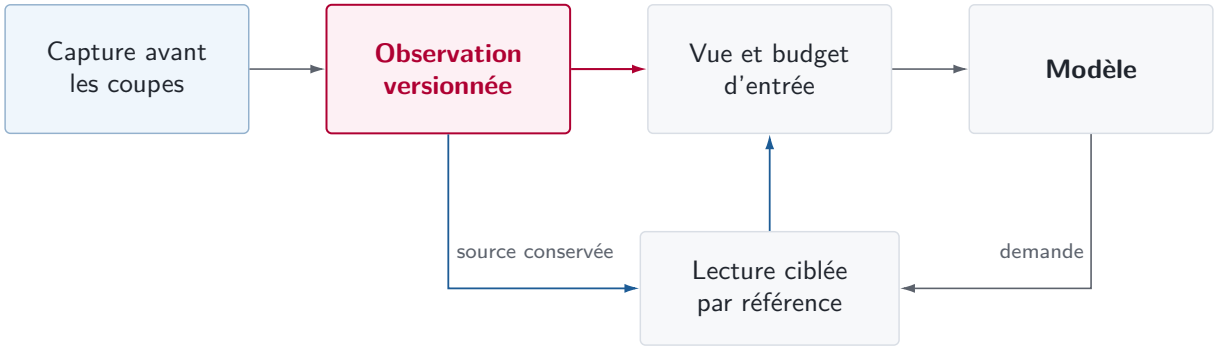


Figure 2 – Architecture proposée, non implémentée. La capture porte son statut de complétude ; la vue indique ses omissions. Une lecture ciblée récupère une source existante, sans rejouer l'action qui l'a produite.

6.1 P0 — Mesurer avant de compresser davantage

Capturer le résultat natif avant filtrage, avec empreinte, taille, encodage, quota et statut complet/incomplet. Le journal référence cet artefact. Chaque vue indique sa source, sa version, les offsets ou chemins JSON retenus, les éléments omis et le motif. Un quota dépassé ne doit jamais produire une déclaration de capture complète.

Pour chaque requête, mesurer séparément instructions, schémas outils, faits, historique et résultats. Distinguer estimation locale, usage retourné, plafond configuré et capacité fournisseur connue ou inconnue. L'admission doit respecter une réserve de sortie dans la fenêtre disponible ; augmenter `max_tokens` peut réduire l'espace restant pour l'entrée. Séparer aussi attente en file et temps fournisseur.

6.2 P1 — Contexte explicite par run et phase

Conserver observations, hypothèses, essais, contradictions et états *candidat, confirmé, inconclusif, erreur, non testé*. Fournir une lecture par référence et cible, limitée par taille. Comparer d'abord une sélection déterministe forte à une sélection augmentée par le modèle. Le découpage par poids des preuves devra préserver les corrélations multi-services.

Un registre de campagne explicite peut remplacer la seule mémoire implicite du dialogue. L'isolation exige soit des dépendances par run, soit une isolation de processus vérifiée. Une répétition d'action peut être légitime ; elle doit être justifiée, attribuée, comptée et admissible selon l'état et la répétabilité de l'action. L'objectif est d'éviter le rejeu involontaire, pas d'interdire toute répétition.

6.3 P2 — Donner au modèle un rôle mesurable

Attribuer chaque décision au scanner, aux règles, au modèle ou au fallback. Si le mémo compact doit modifier la détection, créer des propositions structurées séparées, soumises à vérification. Si aucun bénéfice valide n'apparaît, réserver le modèle à une explication utile ou retirer l'appel du chemin critique. L'assemblage factuel reste disponible sans génération.

7 Protocole proposé : isoler les causes du changement

7.1 A. Fidélité de représentation, hors ligne

Geler un corpus synthétique, puis des traces historiques dont la provenance est vérifiée. Annoter *avant* le test les informations nécessaires : cible, service, résultat positif ou négatif, contradiction et référence. Varier la position début/milieu/fin, l'ordre des services, le volume, les doublons, les erreurs et la dispersion des preuves. La sensibilité à la position et les tâches de combinaison motivent ces facteurs [1, 2] ; elles ne préjugent pas du comportement des modèles actuels.

Bras	**Représentation comparée**	**Interprétation**
R0	Données intégrales lorsque leur taille respecte le budget.	Référence ; exclusions hors budget explicites.
R1	Projection actuelle dans un dispositif expérimental offrant les mêmes lectures que les autres bras.	Effet de la représentation actuelle ; ne reproduit pas tout le runtime sans lecture.
R2	Projection déterministe améliorée, références et omissions.	Effet d'une meilleure sélection sans résumé LLM.
R3	R2, avec résumé LLM additionnel.	Exploration distincte : qualité et coût du résumé.

Table 2 – Ablation proposée de représentation. R0–R3 évitent de confondre ces bras expérimentaux avec les constats C1–C8.

Le modèle, la politique de décision et les plafonds restent identiques. **Tous les bras disposent des mêmes outils de lecture.** Une seconde ablation retire ces outils de tous les bras ; changer leur disponibilité pour un seul bras mélangerait deux facteurs. Le nombre de lectures choisi est une mesure. Un témoin recevant seulement les preuves annotées sert de référence contrôlée, sans être présenté comme système déployable.

7.2 B. Décision et utilité marginale

À preuves initiales communes, comparer un routage à règles, le même routage avec résumé LLM, un agent choisissant ses lectures/actions, puis cet agent avec R2. Les profils compact et full ne sont pas un contraste causal propre : ils changent outils, budgets, obligations, reprises et rôle du modèle à la fois.

Dimension	**Mesures à préenregistrer**
Conservation	Rappel des preuves requises ; omissions silencieuses ; contradictions conservées ; références résolubles.
Attribution	Précision cible/service/référence ; origine des décisions ; nouvelles hypothèses LLM effectivement vérifiées.
Efficience	Coût, tokens et délai par preuve acceptable ; lectures ; actions redondantes ; reprises et échecs.
Robustesse	Stabilité aux permutations et paraphrases ; abstention ; fidélité des justifications revue en aveugle.

7.3 C. Validation ultérieure en laboratoire

Le replay hors ligne ne reproduit pas l'effet d'un choix sur le réseau. Les futurs runs contrôlés exigent une demande explicite, la cible nato/pve-nato, des resets, le même inventaire public et snapshot CVE, des budgets comparables et des versions gelées. Prévoir répétitions par famille et variantes tenues à l'écart des ajustements. Une source commune n'est pas une nouvelle observation indépendante. Un juge LLM ne devient pas seul arbitre de la preuve.

8 Décisions à prendre et limites de lecture

8.1 Critères d'acceptation proposés

Avant le pilote, fixer la marge de non-infériorité, les plafonds coût/délai et l'unité d'analyse. Aucun seuil universel de gain n'est justifié par cette revue. L'hypothèse nulle est admissible : le LLM peut ne pas améliorer la tâche testée.

Étape	Barre d'acceptation à vérifier
Capture	Toute coupe est déclarée ; chaque extrait renvoie à une source disponible ou explicitement incomplète.
Contexte	Budget d'entrée respecté lorsqu'il est connu ; omissions et inconnues visibles ; aucune confusion entre bytes, tokens et sortie.
Reprise	Références résolubles ; pas de rejeu involontaire ; ambiguïtés conservées ; tests d'entrelacement sans mélange A/B.
Utilité	Gain éventuel attribuable aux décisions du modèle, au-delà du scanner et des règles, avec coûts et résultats négatifs inclus.

8.2 Incertitudes à garder ouvertes

Il reste à inventorier quels outils conservent déjà un brut, à identifier quelles familles souffrent réellement des coupes inter-phases, et à vérifier la concurrence possible dans les modes déployés. Une bonne projection déterministe peut suffire ; les lectures ciblées peuvent être lentes ou manquer la source utile. Ajouter un résumé ou des agents peut donc augmenter le coût sans gain.

Les propositions de registre et de chargement à la demande s'appuient sur un retour d'ingénierie [3], pas sur une validation LANCE. Les documents de référence demeurent `docs/architecture/report-writing.md`, `docs/architecture/run-artifacts.md` et `src/agent/phases/README.md`. Les notes, scripts et le journal de validation permettent de retrouver les constats ; cette étude n'atteste aucune nouvelle performance réseau.

8.3 Sources primaires : versions et portions effectivement lues

Consultation le 29 septembre 2026. Revue ciblée, non systématique. Pour Liu et al. et RULER, seules les pages de résumé, auteurs et historique de versions ont été lues, pas les articles complets. Ces travaux portent sur les tâches et modèles de leur époque ; aucun résultat numérique n'est transféré au pipeline.

Pour Anthropic, le corps principal a été lu, notamment les sections sur la composition du contexte, la recherche à la demande, la compression et les notes structurées. Il s'agit d'un retour fournisseur ; ses recommandations doivent être comparées à une solution déterministe et à budgets identiques.

9 Conclusion et prochaines étapes

Les pertes possibles se situent à plusieurs frontières : capture, projection, historique et transmission. Les réparer demande de distinguer une source complète d'une vue abrégée et de rendre chaque omission traçable. Les probes locales caractérisent ces mécanismes, sans établir leur effet sur un audit réel ni désigner le contexte comme cause des timeouts historiques.

La prochaine étape proposée est une mesure hors ligne de conservation et d'attribution, avec sélection déterministe renforcée et lectures disponibles à tous les bras. Une expérience de décision puis une campagne autorisée pourront mesurer l'utilité marginale du modèle. Le succès d'une compilation, d'un test logiciel ou d'un résumé ne constitue pas cette preuve.

Références

[1] N. F. Liu et al. *Lost in the Middle : How Language Models Use Long Contexts*. arXiv :2307.03172v3, 20 novembre 2023 ; publication TACL. Résumé : dépendance à la position en QA multi-documents et récupération clé-valeur.

https://arxiv.org/abs/2307.03172v3

[2] C.-P. Hsieh et al. *RULER : What's the Real Context Size of Your Long-Context Language Models ?* arXiv :2404.06654v3, 6 août 2024 ; COLM 2024. Résumé : benchmark synthétique de récupération, suivi multi-étapes et agrégation.

https://arxiv.org/abs/2404.06654v3

[3] Anthropic Engineering. *Effective context engineering for AI agents*. 29 septembre 2025. Recommandations : références légères, chargement à la demande, notes et compression évaluée sur la conservation d'informations utiles.

https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents